Advances in Cognitive Systems X (20XX) 1-6 Submitted X/20XX; published X/20XX

A Computational Model of Anticipatory Thinking

Jeff Aguilar JAGUILAR@ARA.COM
Kevin Volkel KVOLKEL@ARA.COM
Rodney Metoyer RMETOYER@ARA.COM
Adam Amos-Binks AAMOSBINKS@ARA.COM
Applied Research Associates, Inc., 4300 San Mateo Blvd NE, Albuquerque, NM 87110

Abstract

Agents that perform well in testing can still fail catastrophically when deployment conditions shift. We argue that agents should manage this risk the way scientists and analysts do: by running thought experiments to stress-test their own solutions. We introduce a computational model of anticipatory thinking (AT) in which an agent uses prospection to predict how its solver will perform, creativity to generate task variations where it is predicted to fail, and adaptation to fix those failures while preserving performance on the original task. We implement this model of AT in a reinforcement learning agent using a world model. Our AT-enabled agent yields greater performance on kinematic stress tests and unseen track widths in CarRacing-v3 than Dreamer and PPO baselines. Notably, our agent also had significantly greater overall performance than the baseline agents, rather than simply trading overall performance for better performance in the stress tests. These results encourage us to pursue agent designs that actively construct tasks where they are likely to fail, rather than waiting to encounter them, so they are better prepared for the long tail of errors. Because each step of the computational model of AT can be instantiated and tested independently, the model offers a general, testable foundation for building risk management into agents beyond reinforcement learning.

1. Introduction

Anticipatory thinking (AT) uses thought experiments to reveal insights about how the future might unfold. These insights address the need for effective risk management when real-world conditions foil the best preparations, often referred to as the long-tail-of-errors. Managing risks in the long-tail-of-errors has many real-world applications ranging from safety-critical experiments designed by scientists to mission-critical operations executed by intelligence analysts. Successful AT requires prospection, creativity, and adaptation at demanding levels of rigor and vigilance prone to human error, creating a decision support opportunity for AI systems.

However, AT in AI systems still lag even error-prone human ability. AT has been formulated as a form of meta-cognition in a goal-reasoning agent in a narrow application, lacking the generalizability of a computational model that can be operationalized in other representations. An AT assessment framework, designed to reveal variations and novel risks, lacks interventions to address the risks. Key developments in world models, agents that learn to simulate the dynamics of a task environment, have led to agents that impressively generalize across tasks, yet do not ex-

© 20XX Cognitive Systems Foundation. All rights reserved.

plicitly manage risks. Despite progress in AT and adjacent fields, explicitly managing risk in the long-tail-of-errors remains a barrier for AI systems providing decision support to mission and safety critical tasks. A generalized form of AT that is explicit in AI systems would provide a customizable risk-management capability with the assurances needed in safety and mission critical applications.

We overcome these limitations by formulating AT as an explicit agent capability applicable to a wide-range of tasks in need of effective risk management. Our thesis is that, if AI agents can reason about tasks like a scientist or special operations analyst performing thought experiments on their primary task, simulating the outcome, and adapting their solution to account for any new risks, then AI systems will manage risks effectively enough to support mission and safety critical domains. This approach led to three contributions. First, we define a computational model of AT. We decompose AT into three distinct steps –prospection, creativity and adaptation – and provide formal definitions of their relationships along with a running concrete example. This explicit formulation of AT enables generalization across automated decision making methods. From classical planning to reinforcement learning to language models and perhaps combinations of them, the individual steps of AT can be designed and evaluated to provide the assurances needed in safety or mission critical application. Second, we operationalize our computational model of AT in a reinforcement learning agent. Without loss of generality to other automated decision making methods, we train our AT agent to learn a world model (prospection), from which it conducts systematic thought experiments that reveal where it fails (creativity), and then adjusts its policy accordingly (adaptation). Finally, our experimental results show that AT supports good risk management in the long-tail-of-errors. We train our AT agent and baseline agents to navigate race tracks in Gymnasium's Car Racing environment with an added twist: we adjust track widths from training to deployment settings. When our AT agent encounters new track widths, it deliberately considers alternatives and adapts, managing the risk of these new conditions while ensuring it solves the original task as posed. The results show our AT agent is both more robust to unseen track widths and improves performance overall compared to baselines. Interestingly, when baseline agents are trained on the same alternatives our AT agent generates and then adapts to, the performance lags our AT agent, indicating the AT training regime improves performance. We conclude with how our computational model of AT can be applied to additional task types, support mission and safety critical tasks, and operationalize alternative methods of prospection, creativity, and adaptation.

2. Computational Model of AT

Our goal is to endow AI agents with the capability to explicitly manage risk in mission and safety critical action-oriented tasks, what we call AT. AT enables agents to go beyond their learned or engineered knowledge acquired at design/test time and be resilient to risks they encounter at deployment time. We break AT down into three key steps. First, prospection allows an agent to simulate and predict how their problem solving ability (their solver) will perform on any given task. This differs from typical approaches that simply apply their solver directly to a task in a real-world environment. Second, an agent creates novel variations of a critical task where prospection predicts its current solver cannot solve. We refer to this as a new risk in need of mitigation. Finally, an agent adapts its solver to guarantee that it can solve the original task and generated variations. We bring

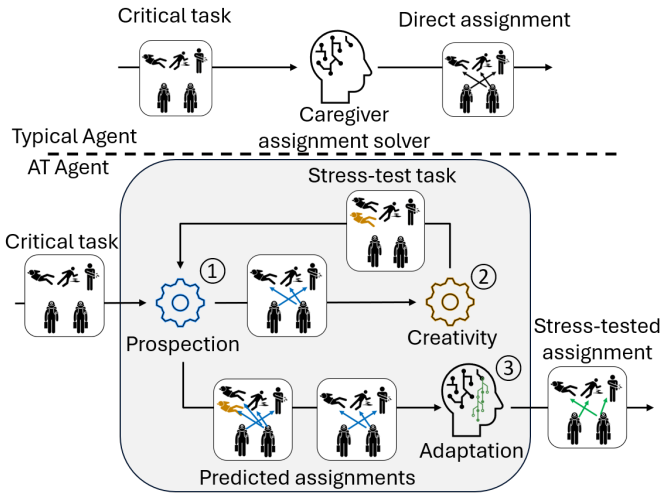


Figure 1. In the top pane, a typical agent applies their solver directly to a critical task without managing any long-tail-of-errors risks and may fail catastrophically. In contrast, an agent who applies a computational model of AT will predict the outcome of applying its solver (Prospection), generate variations of the task called stress-tests (Creativity), and adapt its solver to the predicted results of the critical task and stress-tests (Adaptation), resulting in a stress-tested assignment.

these three steps together into a single AT procedure by walking through an illustrative example of a safety-critical task.

Our safety-critical task is from combat casualty care (CCC), where an agent must assign caregivers to battlefield casualties such that lives are preserved. The crux of this task is to match caregiver skills and supplies to casualty needs. Assigning an over-qualified caregiver decreases the clinical capacity to treat other casualties, putting their lives at risk, whereas an under-qualified caregiver endangers their assigned casualty. Effective decision support requires the agent to manage risks around its caregiver assignments in a partially observed environment with noisy returns from actions that sometimes do not execute as planned.

Definition 1 (Task) A tuple, $\tau\langle \mathcal{P}, thresh, A \rangle$ is a set of task-defining free parameters $\mathcal{P}$ considered solved when threshold, $thresh$, is reached or exceeded by executing an action sequence from actions in A.

We refer to tasks that must be solved in a safety- or mission-critical deployment setting (i.e., post development) as ***critical tasks***, τ^{crit}. An example critical task in the CCC domain is in Figure 1 with two caregivers and three casualties. The task is considered solved (i.e., reaching $thresh$) if all lives are preserved.

Definition 2 (Solver) is a function σ applied to a task τ that generates an action sequence in an attempt to achieve $thresh(\tau)$

In Figure 1, the typical agent applies its solver σ to the task directly, producing the action sequence A^{crit} that aligns the two caregivers whose skills will meet the three casualties' needs.

Procedure 1 An Anticipatory Thinking procedure that identifies risks when solving a critical task and adapts its problem solving approach to mitigate these risks.

Require: a solver σ, a critical task τ^{crit} and mitigation level $risk$
Ensure: a modified σ' with guaranteed performance on task τ^{crit} and mitigated risks

procedure ANTICIPATORY THINKING(σ, τ^{crit}, $risk$)
  Let $\widehat{y}^{crit} \leftarrow \pi^{crit}$ ▷ **Prospection** - Predict results of solving the critical task
  if $\widehat{y}^{crit} > thresh(\tau^{crit})$ **then** ▷ If solver predicted to solve critical task
    while $|\mathcal{T}| < risk$ **do** ▷ Create a enough stress tests
      Let $\tau^{stress} \leftarrow \chi^{crit}$ ▷ **Creativity** - Generate a stress task, critical task variation
      Let $\mathcal{T} \leftarrow \tau^{stress}$ ▷ Add a set of stress tests
    $\sigma' \leftarrow \alpha^{\sigma}_{\mathcal{T}}$ ▷ **Adapt** - a new solver that solves the critical task and stress tests
  return σ'

Since this CCC task is safety-critical, there is a need for an AT-based agent with the ability to consider how the world may evolve without actually interacting with it. This breaks with typical approaches that simply apply their solver directly to achieve the assignment in the top pane of Figure 1. In tradition with the cognitive science convention (Szpunar et al., 2014), we refer to this future-oriented capability as prospection.

Definition 3 (Prospection) of a generic task τ indexed by k as τ^k is tuple $\pi^k = \langle \sigma, \widehat{\mathcal{A}}, \widehat{\mathcal{O}}, \widehat{y} \rangle$ and is the agent predicting the solver σ applied to τ^k resulting in predicted observations $\widehat{\mathcal{O}}$ from executing action sequence $\widehat{\mathcal{A}}$ with result $\widehat{y}$. Prospection occurs entirely in simulation without interaction with the real world.

In Figure 1, our agent uses prospection to simulate how its solver will change the world state from its actions, rather than simply applying it directly to the environment based on historical or known examples. After each simulated caregiver assignment action, our agent has insights into how the world has changed. This deliberate exploration of the future invites a type of thought experiment we refer to as `stress-test tasks`, or creative variations on τ^{crit} resulting in new tasks that are difficult to solve. These stress tests are framed with the standard definition of creativity (Ritchie, 2001), where new tasks are both novel and effective, specifically revealing risks in need of mitigation. An agent's creativity is the source of stress-tests:

Definition 4 (Creativity) applied to a critical task τ^{crit} results in a tuple, $\chi^{crit} = \langle \tau^{stress}, \sigma \rangle$, with some new task τ^{stress} such that the parameters between the critical and stress tasks differ $\mathcal{P}(\tau^{crit}) \neq \mathcal{P}(\tau^{stress})$ and the predicted result of τ^{stress} using solver σ does not meet the solved threshold of the critical task, $\widehat{y}(\pi^{stress}) < thresh(\tau^{crit})$.

Our CCC agent uses its prospection and creativity to conceive of stress tests where an additional casualty is discovered at a later time. Since the medic is ill-equipped to handle both casualties, it will be impossible to reach $thresh$ and ensure all casualties survive. This new task, and many others, are sampled from the long-tail-of-errors, presenting an opportunity for the agent to adapt its solver and more effectively manage these risks:

Definition 5 (Adaptation) modifies an agent's solver σ and is represented with a tuple, $\alpha = \langle \sigma, \sigma', \tau^{crit}, \tau^{stress} \rangle$, such that σ' solves critical task τ^{crit} and stress test τ^{stress} that σ cannot.

In Fig. 1, our agent adapts its solver to account for the risk of uncovering the additional casualty. It solves the original task as well as the new stress test, mitigating this risk. To further sample from the long-tail-of-errors, our agent generates additional stress tests, adapting its solver to provide the high assurances needed in this safety-critical domain. We combine these three steps into an AT procedure in Procedure 1, noting that adapting the solver is done in a batch manner for all stress tests $\mathcal{T}$, rather than adapting for each stress test, for easier readability. In Section 3.2 we operationalize this abstract model into a reinforcement learning agent that uses AT to sample from the long tail of errors, improving its risk management and, surprisingly, its overall performance.

3. Evaluation

Our evaluation tests whether an agent that applies AT manages risk more effectively under task variation. This capability is key to managing risk in the long-tail-of-errors encountered in real world deployments for mission- and safety-critical tasks where assurances are needed. We operationalize our computational model of AT as a reinforcement learning (RL) agent and evaluate its ability to anticipate and adapt to nearby tasks outside its training distribution. We pivot from the safety-critical CCC running example to the well-studied `CarRacing-v3` environment where comparisons between existing RL approaches and AT are easily made. Risk in `CarRacing-v3` are the track segments where the RL agent incurs the greatest loss (e.g. tight turns on narrow roads).

3.1 Domain

We use Gymnasium's `CarRacing-v3` environment, a continuous-control benchmark in which an agent navigates procedurally generated tracks, earning rewards for visiting new track tiles while incurring a small penalty at each timestep before completion. An episode ends when the agent traverses all tiles or remains off track for too long. The environment has a high-dimensional observation space and requires precise control actions to solve tasks (track), well-suited for AT. Car Racing has also been used to baseline several world-model approaches. Since we use a world model for prospection, this makes it a natural testbed for AT.

We expose controllable parameters in its procedural track generator, including track width, turn rate, and track radius. We select track width as the primary task parameter because it provides a continuous and interpretable means of changing the precision required for successful driving while preserving the underlying objective and track-generation process. Prior to implementing AT, we first characterize the effect of this parameter by training a standard Dreamer (Hafner et al., 2025) RL agent for 92k gradient updates using only the default track-width scale of 1.0, then evaluate the resulting agent across width scales from 0.4 to 1.6 on a held-out set of procedural-generation seeds, using the same seeds at every width.

As shown in Fig. 2, the agent achieves relatively high but variable reward at its training width. Performance degrades as tracks become narrower, whereas wider tracks generally yield higher and more consistent rewards due to greater tolerance for control errors. This structured performance

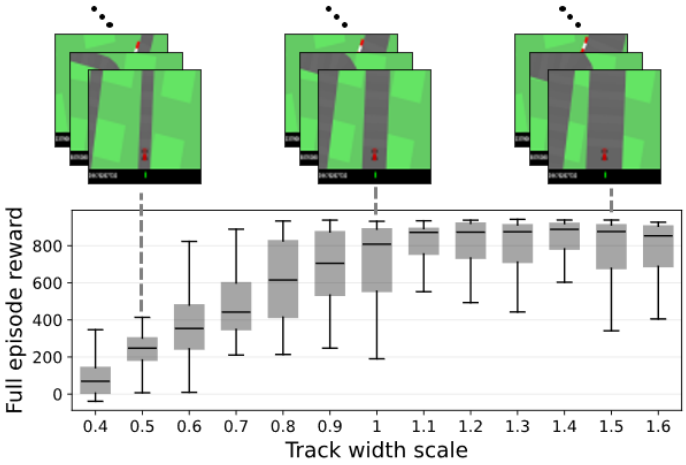


Figure 2. Track width as a lever for difficulty. Evaluation of episode reward vs. track width on a Dreamer agent trained with default procedural track generation parameters (track width scale = 1.0) demonstrates track width to be a controllable substrate for varying task difficulty.

gradient exposes a clear competence boundary around the training distribution, making width a suitable parameter for generating nearby, semantically consistent task variants – stress tests – that challenge the RL agent's current policy (solver).

3.2 Procedure

Our operationalization of an AT agent in RL is depicted in Figure 5. The crux of our approach is using a world model for prospection. This enables the agent to predict its performance on a critical task, rather than just applying its policy directly (in RL, the solver is a policy), and identify stress tests by evaluating critical task variations. **`The world model is an agent's substitute for interacting with the actual training environment.`**

Agent Design: We implement our AT RL agent using a Dreamer based agent design (Hafner et al., 2025).[1] Dreamer consists of three components: (1) a recurrent world model that encodes observations into latent states and predicts future states and rewards given the agent's actions; (2) a critic that estimates future discounted returns from latent states; and (3) an actor that selects actions to maximize those returns. The world model is trained both to predict future dynamics and to reconstruct observations from its latent representation. Together, these components enable AT prospection: beginning from latent states inferred from replay, the world model rolls the current actor forward in the agent's internal representation of the environment, and the predicted rewards and critic values are combined into λ-returns that estimate future performance. The world-model

1. Our experiments build on Natural Dreamer (InexperiencedMe, 2025), a compact PyTorch implementation of DreamerV3. While Natural Dreamer omits certain stabilization mechanisms present in the full DreamerV3 algorithm (Hafner et al., 2025), its compact, modular design enabled us to instrument imagined rollouts and expose their predicted rewards and λ-returns, while modifying replay sampling and the environment-interaction loop to integrate AT.

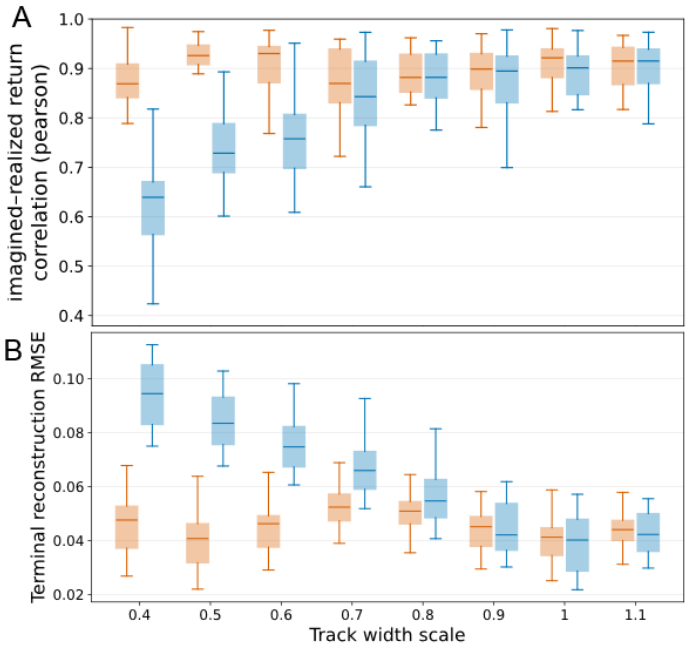


Figure 3. Prospection Performance. Dreamer trained on equal proportions of 0.5- and 1.0-width tracks (orange), compared with Dreamer trained only on default-width tracks (blue). (A) Correlations between prospection λ_0 and realized short-horizon discounted return and (B) terminal reconstruction RMSE both improve on narrower and several interpolated track widths when 0.5-width tracks are included in training.

community typically describes this latent generation as "imagination". We use the broader term prospection due to the critical role of this process in anticipatory thinking.

World-Model Reliability: We evaluate the reliability of this world model as a prospection mechanism by comparing two Dreamer agents: one trained only on 1.0-width tracks and another trained equally on track widths 0.5 and 1.0. From fixed starting locations on matched tracks, we compare world model λ_0 values (λ-returns at the start of prospection rollouts) with real short-horizon returns (Fig. 3A), measure root mean squared error (RMSE) between the final reconstructed latent frame and the corresponding real frame (Fig. 3B), and visually inspect reconstructed prospection trajectories (Fig. 4). These analyses indicate reliable world model prospection when the training distribution covers the environments being evaluated.

Agent Training: The three components are trained concurrently using separate objectives. The world model learns the environment dynamics from real trajectories stored in replay, whereas the actor and critic are trained primarily on latent trajectories generated by the current world model. The critic is trained to predict bootstrapped λ-returns, and the actor is optimized to select actions that maximize those predicted returns. Gradients from the actor and critic objectives do not update the world-model parameters. Consequently, newly collected experience expands the predictive coverage of the world model while also changing the prospection trajectories on which the actor and critic are trained.

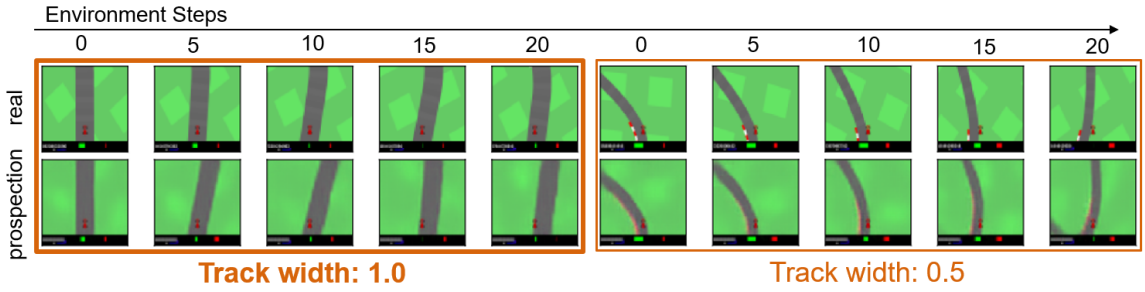


Figure 4. Example real vs reconstructed prospection trajectories for a 1.0-width track (left panel) and 0.5-width track (right panel) using Dreamer trained on equal proportions of 0.5- and 1.0-width tracks (orange traces in Fig. 3).

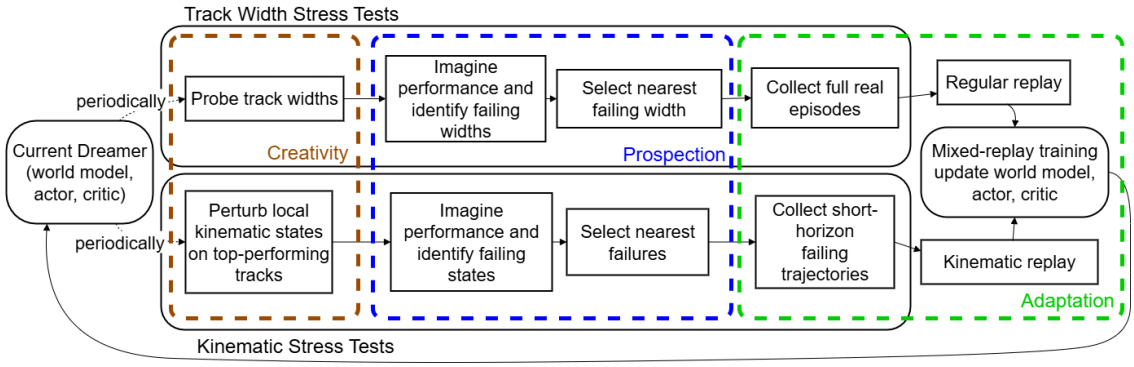


Figure 5. Integrating anticipatory thinking into Dreamer learning loop.

AT Implementation: We implement AT in `CarRacing-v3` as a periodic outer loop around Dreamer training (Fig. 5), mimicking how an agent would train/test and then encounter critical tasks during deployment. We initialize Dreamer with a warm-up phase of training on default-width tracks (track-width scale 1.0), where the world model learns a reasonable latent representation of the environment. Afterwards, the AT loop performs two complementary stress tests: track-width and kinematic stress testing, each proceeding through the creativity, prospection, and adaptation steps in our computational model of AT. During creativity, the agent proposes nearby variations on the critical task by perturbing track width (Fig. 2) or local kinematic conditions (e.g. lateral position, heading, and speed, see Fig. 6) on well-performing states. During prospection, each candidate perturbation is instantiated briefly in the Gymnasium environment to obtain a first observation, which is then encoded into a latent state, and then rolled forward with the current policy using the world model's prospection. Critic-derived λ-returns estimate the expected performance of each candidate, with low-return candidates treated as likely failure-inducing variations on the critical task–ideal stress-tests to which the AT agent adapts. During adaptation, the resulting stress-tests are simulated in the real environment, and the resulting trajectories are added to replay so that subsequent Dreamer updates can improve performance in those regions of the task space.

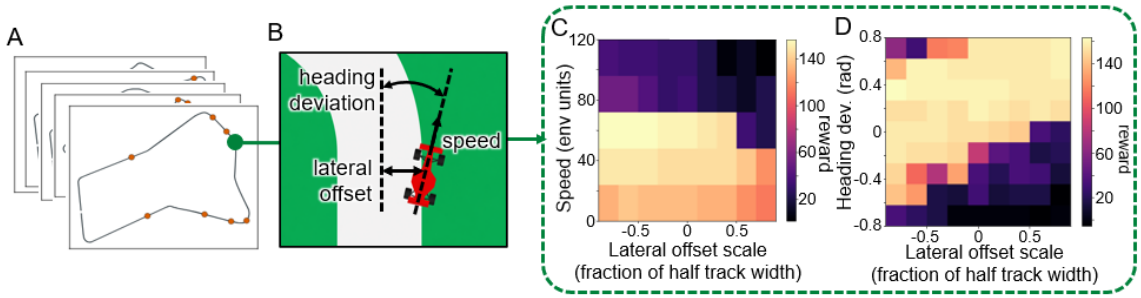


Figure 6. Creativity for kinematic stress tests. (A) We first identify top full-rollout reward tracks from a batch of newly sampled unseen tracks. We then sample geometrically relevant positions along each track. (B) For each position, we perturb the starting relative position and kinematics. Sample heatmaps of short-horizon reward vs (C) speed and lateral offset (with 0 heading deviation) and (D) heading deviation vs lateral offset (with a starting speed of 60 env units) indicate large variation in real short horizon performance along these perturbation dimensions

Track Width Stress Tests: Recall from Fig. 2, track width is an effective dimension on which to stress the AT agent. After the warm-up training phase, we present it with critical tasks every subsequent 4000 gradient steps. The initial task must first satisfy a predefined reconstruction-loss criterion, reducing the likelihood that subsequent failures simply reflect an immature world model rather than sensitivity to track width. We then probe a bounded neighborhood around the active width, the beginning of the `creative` AT step. Each width is evaluated using 100 short prospection trajectories from unique unseen track seeds (using an identical seed set across track widths). Our `prospection` step divides failures into representation failures (identified by mean reconstruction loss above a predefined reconstruction-loss criterion) and policy failures (identified when the normalized critic-derived λ-return is below 80% of the active-width value). A prospective policy failure must also achieve a mean full-simulation score below 80% of the active-width score over 50 validation episodes. Candidate widths are examined nearest-first; when a width satisfies both failure criteria, the representation failure takes precedence. Once a failing width is promoted, it becomes the active `adaptation` target for new full-episode collection. To ensure performance on past widths, replay is maintained according to a staged distribution: 50% active, 25% default, 15% nearest mastered width, and 10% distributed across other mastered legacy widths.

Kinematic Stress Tests: Following the initial warm-up period, we perform periodic (every 2000 gradient updates) kinematic stress tests. The `creative` step of AT generates variations of well-solved trajectories to identify nearby initial conditions under which the current policy is predicted to fail. By holding the track and local geometry fixed while varying the car's initial kinematic state, these tests expose plausible failures before they arise in a deployment. Figure 6 shows that these perturbations produce a broad range of short-horizon returns, including nearby transitions from successful to failed behavior. We select at most two widths according to their target shares in regular replay and evaluate 20 unseen procedural-generation seeds per width. We retain five highest scoring tracks as critical tasks, preserving width target shares in replay. For each critical task, we select eight locations that span straight segments, curve entry, curve apexes, and curve

Table 1. Agents evaluated for performance across track widths and stress tests.

RL Agent	**Brief description**
AT-Dreamer	Dreamer agent trained with anticipatory thinking (312k gradient steps).
Baseline-Dreamer	Dreamer agent without anticipatory thinking trained on default 1.0 width tracks (314k gradient steps).
PPO Baseline A	CNN-based PPO agent trained uniformly on all AT-generated tracks (5.8M environment steps).
PPO Baseline B	PPO Baseline A after extended training (17.54M environment steps).
PPO Baseline C	PPO agent using grayscale two-frame stacking and frame skip, after extended training (18.6M environment steps).
PPO Baseline D	PPO Baseline C trained with a mixture regular and short-horizon kinematic scenarios similar to AT-Dreamer's 20% sampling distribution (13.2M environment steps - training collapses beyond this point).

exits. At each location, we evaluate a Cartesian grid over the starting speed, heading deviation, and normalized lateral offset from the centerline.

We apply `prospection` on each candidate by first briefly executing 10 real-environment steps to obtain a grounded recurrent latent state. From this state, the current actor is rolled forward for 15 steps in the world model, and the critic-derived λ-return at the beginning of the predictive trajectory is used as the candidate's prospective performance estimate. Figure 3A shows a strong association between prospection λ-returns and realized short-horizon returns, supporting the use of λ-return as a prospective indicator of real-environment performance. The candidate with the highest predicted λ-return defines a local successful reference. We calibrate a prospective failure threshold using 128-step real-environment rollouts sparsely sampled across the distribution of prospection returns. Candidates in the bottom 20th percentile of realized return are labeled as real failures, and the largest predicted λ-return among these failures is used as the prospective failure threshold. Candidates below this threshold are treated as likely failures and are prioritized by closest normalized kinematic distance from the local successful reference, with predicted λ-return used as a secondary ranking criterion. This favors nearby failures that require only small perturbations of an otherwise successful initial condition. `Adaptation` to these failures uses real-environment rollouts stored in a separate kinematic replay buffer. We retain only 128-step sequences to target the localized short-horizon failures. Subsequent Dreamer minibatches contain 20% kinematic replay and 80% regular replay. This mixture allows Dreamer to adapt to newly identified failure conditions without training exclusively on failures or degrading performance on previously successful tasks.

3.3 Results

We evaluate the performance of our AT-Dreamer agent against a range of baseline agents (Table 1). In the most direct comparison, we contrast our AT-Dreamer agent with a Dreamer agent trained without AT for a comparable number of gradient updates on only 1.0-width tracks. The remaining baselines are variants of a standard Stable-baselines3 (Raffin et al., 2021) PPO with a CNN policy,

Table 2. We compare AT-Dreamer against baseline agents using Welch's t-test to test if stress-test performance is drawn from the same distribution and and Cohen's d to measure the magnitude of any difference. The p(raw) values are significant at the 0.05 alpha level and so we reject the null hypothesis that AT-Dreamer performance is from the same distribution as other agents. In the Cohen's d column we find the magnitude of the difference between medium and large ($0.5 < 0.67 < 0.8$) when comparing against Baseline-Dreamer and very large (>1.2) against all other agents.

Agent comparison	mean diff	t	df	p (raw)	Cohen's d
AT-Dreamer vs Baseline-Dreamer	17.64	7.36	464.1	8.29e-13	0.67
AT-Dreamer vs PPO Baseline A	29.56	13.93	476.6	2.97e-37	1.27
AT-Dreamer vs PPO Baseline B	43.75	22.61	445.9	5.78e-76	2.06
AT-Dreamer vs PPO Baseline C	39.84	23.2	348.2	1.21e-72	2.12
AT-Dreamer vs PPO Baseline D	35.71	19.14	422.0	3.15e-59	1.75
AT-Dreamer vs All (n=1200)	33.30	19.95	326.7	1.71e-58	1.48

providing all the track seeds used during AT-Dreamer's training (including initial starting conditions for kinematic stress tests). We analyze the results as two separate statistical hypotheses. The first tests whether AT-Dreamer performance differs from the baseline agents in stress-tests where the risk is greatest and primary motivation for integrating AT. The second tests whether there is an overall performance difference across whole tracks between AT-Dreamer and baseline agents; improved performance in stress-tests should not detract from overall performance.

Stress-test performance differences Figure 7 contains the short horizon reward performance of baseline agents on kinematic stress-tests that are generated from new track seeds unseen in training. These stress-tests are similar to those generated during AT training. Generally, AT-Dreamer outperforms other agents across track widths and track geometries evaluated with the mean reward 79.75 and std=23.87 (Baseline-Dreamer was the next highest with mean=62.11, std=28.43). For 1.5-width tests, Baseline-Dreamer exhibits comparable aptitude to AT-Dreamer, likely to due to wider tracks being an easier task. Conversely, on narrower tracks, while all agent's performance degrades, there is greater separation between AT-Dreamer and the other baselines.

Our statistical hypothesis is that AT-Dreamer short-horizon rewards are not drawn from the same distribution as other baseline agents. We first test normality and variances across the six agents with Shapiro-Wilk and Levene's test. Only AT-Dreamer (Shapiro W=0.9891, p=6.72e-02) is consistent with normality while the other five groups fail the Shapiro–Wilk (alpha=0.05), and Levene's test rejects equal variances (W=42.66, p=4.54e-41). We account for these results by using Welch's t-test since it does not assume normality and equal variances. Results of the Welch's t-test, two-sided comparing AT-Dreamer against individual agents as well as all other agents pooled are in Table 2. In the p(raw) column, values are all below an alpha threshold of 0.05, rejecting the null hypothesis that the pairs are drawn from the same distribution. Further, Cohen's d, which calculates how many standard deviations two means differ by, shows a medium-to-large effect size (0.5-0.8) when AT-Dreamer is compared to Baseline-Dreamer, and a very large effect size ($1.0\leq$) when compared to other baselines.

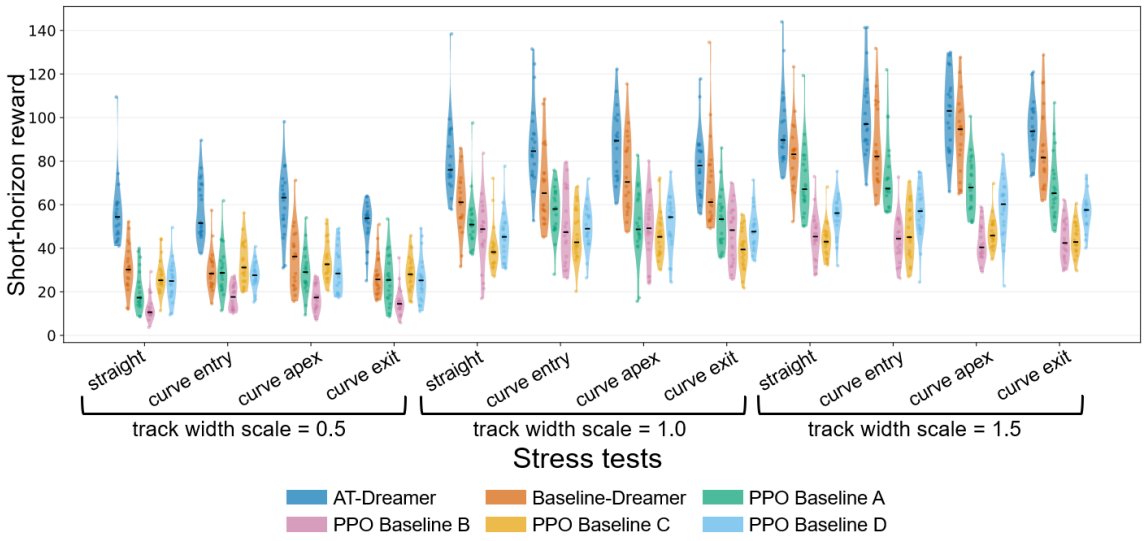


Figure 7. Stress test performance. Violin plot comparison for AT-Dreamer against baseline agents of short horizon rewards vs different stress tests. Two track locations are selected per stress test category (straights, curve entry, curve exit, and curve apex at widths of 0.5, 1.0 and 1.5) for each of 10 unique track seeds unseen in training. At each location, short horizon rewards are averaged across 10 randomized local perturbations of initial speed/heading/lateral offset relative to top performing initial states.

Overall performance differences Next, we evaluate each trained agent on unseen track seeds across a sweep of track widths that we show in Fig. 8, this differs from evaluating performance only on the most challenging portions of a track. AT-Dreamer achieves the greatest full-episode reward across all widths (mean=822.3) and exhibits lowest reward variance(147.6). In extremely wide tracks (widths of 1.4, 1.5, 1.6), the performance gap between AT-Dreamer and PPO Baseline B becomes more narrow, presumably because the task is easier overall, allowing this less robust agent to achieve higher performance than for narrow tracks. The Baseline-Dreamer agent performs comparable to AT-Dreamer near the default training width of 1.0, but Baseline-Dreamer performance deteriorates on both narrower and wider tracks, indicating weaker robustness to changes in the task parameter. The PPO Baseline A agent, which was trained using the same number of real-environment interactions as AT-Dreamer, performs worst across evaluated track widths. Yet, while this result is consistent with greater environment-sample efficiency reported for Dreamer-style agents (Hafner et al., 2019), the performance of subsequent PPO variants remains less than AT-Dreamer across most track widths despite training on more overall environment interactions. Finally, in the widths of 0.4 and 1.6 which lie just outside the range encountered during training, AT-Dreamer retains the strongest performance at both widths. This suggestes a learned robustness and ability to extrapolate beyond its training distribution. Our statistical hypothesis is that AT-Dreamer's full-episode rewards are not drawn from the same distribution as other baseline agents. In an identical analysis to short-horizon rewards, we first test normality and variances across the six agents with Shapiro-Wilk and Levene's test. All agents fail to reject the null hypothesis of the Shapiro–Wilk test at the alpha=0.05 level and are assumed to be not normal. Levene's test rejects

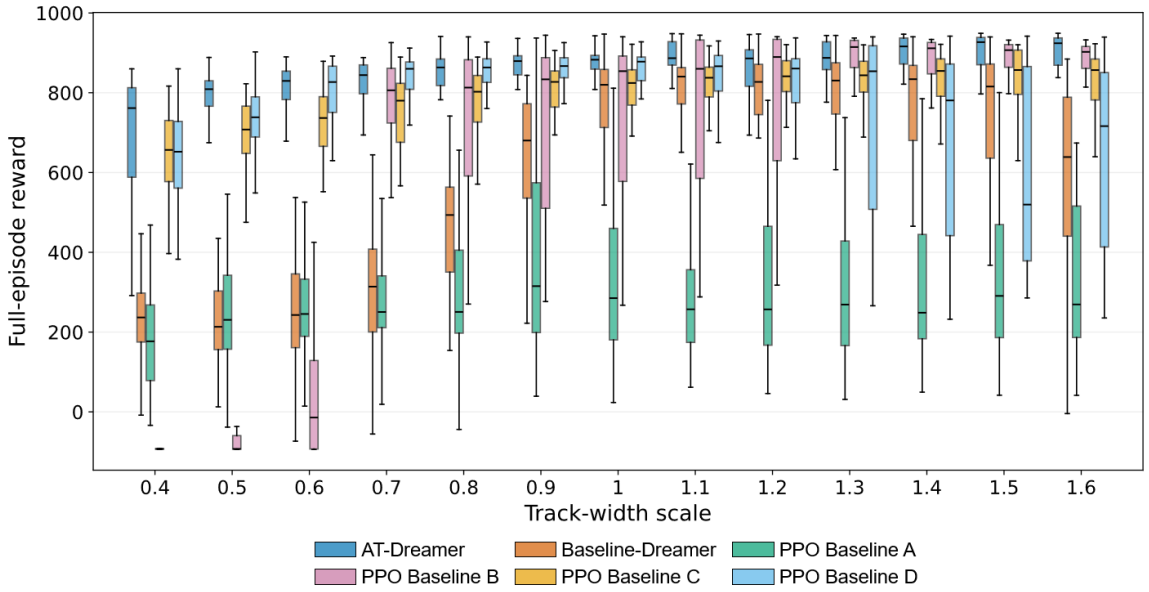


Figure 8. Full episode performance vs track width. Box plot comparison for AT-Dreamer against baseline agents of full-episode reward vs track width. 50 unique track seeds unseen in training are evaluated for each agent and track width.

equal variances (W=162.74, p=1.61e-157) and variances assumed to be unequal. We account for non-normality and unequal variances by using Welch's t-test. Results of the Welch's t-test, two-sided comparing AT-Dreamer against individual agents as well as all other agents pooled are in Table 3. In the p(raw) column, values are all below an alpha threshold of 0.05, rejecting the null hypothesis that the pairs are drawn from the same distribution. Further, Cohen's d column shows a large effect size or greater (Cohen's d $\geq$ 0.8) when AT-Dreamer is compared to all other agents except PPO Baseline C and PPO Baseline D where it is small to medium (Cohen's d 0.2-0.5).

3.4 Discussion

Our computational model of AT is designed to ensure agents more effectively manage the risks in the long-tail-of-errors. We operationalized the model we defined in Section 2 showing that an RL agent using AT is more effective at managing risk (obtaining a greater reward) in the long-tail-of-errors. Our AT-Dreamer agent had the greatest overall mean short-horizon performance among all the agents when tested on unseen track widths and kinematics. Further, Welch's t-test confirmed a statistically significant difference in performance for our AT-Dreamer in these difficult (risky) track segments.

These results suggest that incorporating stress-tests into the AT training loop (Fig. 5) provides the ability to effecetively manage risk in the long tail of errors. The agent periodically challenges itself by predicting its own performance – using its prospection – on small variations to a critical task, and subsequently targeting training toward minimally perturbed tasks that indicate catastrophic

Table 3. We compare AT-Dreamer against baseline agents using Welch's t-test to test if long-horizon performance is drawn from the same distribution and and Cohen's d to measure the magnitude of any difference. The p(raw) values are significant at the 0.05 alpha level and so we reject the null hypothesis that AT-Dreamer performance is from the same distribution as other agents. In the Cohen's d column we find the magnitude of the difference is large or very large (> 0.8) when comparing against all but two agents where the magnitude of the difference is small (<0.5)

comparison	mean diff	t	df	p (raw)	Cohen's d
AT-Dreamer vs Baseline-Dreamer	260.8	21.4	997.6	5.91e-84	1.2
AT-Dreamer vs PPO Baseline A	523.2	55.9	1229.3	0.00e+00	3.1
AT-Dreamer vs PPO Baseline B	220.8	13.6	834.1	3.48e-38	0.8
AT-Dreamer vs PPO Baseline C	38.3	5.5	1138.4	4.89e-08	0.3
AT-Dreamer vs PPO Baseline D	72.8	7.8	1230.6	1.50e-14	0.4
AT-Dreamer vs All (n=3250)	223.2	28.5	1912.2	3.60e-149	0.8

failures, so it may adapt and not succumb to the long-tail-of-errors. This shift in training distribution towards failures is why the AT agent exhibits superior short-horizon performance (Fig. 7) on precisely those categories of risks.

A surprising upside to our original goals is improved risk management allows for adaptation beyond the original critical tasks. Not only does our AT-Dreamer agent improve on short horizon stress tests, it also improves full-episode performance on new tracks and track widths unseen during training (Fig. 8) with the highest overall mean reward. The confirmed statistical differences between AT-Dreamer and baseline agent performance on full-episodes suggests that learning to handle challenging local track geometries enables policies beneficial to long-horizon performance on new tracks.

Lastly, the large magnitude between the high-performing results of AT-Dreamer and the PPO baseline agent performance (Cohen d greater than 0.8 in Table 3, suggests that blind exposure to difficult scenarios is insufficient. Despite being exposed to the very same tracks and challenging starting points, there remains a performance disparity. Our hypothesis is that systematic, curriculum-based targeted stress-tests enables continued performance gains and are necessary to become robust to the long-tail-of-errors.

4. Related Work

Our computational model builds on work at the intersection of anticipatory thinking, world models, and continual learning. Advances in these domains collectively represent the theoretical and technical underpinnings that highlight the need to identify and manage risks ahead of time and enable the ability to autonomously forecast and adapt to these risks.

Anticipatory Thinking: Anticipatory thinking is the core philosophy that we operationalize in this work, originally cast as a human capability (Klein et al., 2011), differentiating risk identification and preparation from merely predicting likely outcomes. Geden et al. (2019) define AT more formally as the explicit search of alternative system states and measure its generative abilities by the

diversity, uniqueness, specificity, and remoteness of anticipated futures. Such metrics can be traced to the broader concept of prospection, which Szpunar et al. (2014) decompose into simulation, prediction, intention, and planning modes. Schacter et al. (2017) further explore a particular form of prospection that draws on personal experience, and shows its benefits in supporting many human cognitive functions including long-term decision-making, emotion regulation, and planning.

Recent advancements directly link these human functions to computational risk management and support. Scenario Explorer – a platform developed for our safety critical task of combat casualty care – assists analysts in constructing, combining, and evaluating possible futures to reduce surprise (Argenta et al., 2020). Amos-Binks & Weber (2023) quantify risk-management in a real-time strategy game context by using causal inference to characterize agent mitigations as either anticipatory or reactionary. Amos-Binks & Dannenhauer (2020) implement AT as a metacognitive goal-reasoning agent that critiques its own plan for vulnerabilities and makes informed adjustments. And Amos-Binks et al. (2023) introduce an assessment paradigm for AI agents that generates stress-tests for preparedness against both unseen and rarely seen failures. These works demonstrate the value of operationalizing key anticipatory thinking concepts; our work extends this by defining AT as a general cycle of prospection, creativity, and adaptation and demonstrates the complete cycle in a learning agent.

World Models enabling prospection: World models learn predictive representations of environmental dynamics that support understanding, simulation, planning, and decision-making (Ding et al., 2025). Computational AT is made practical by the advent of these prospective abilities. Ha & Schmidhuber (2018) demonstrate that a controller can effectively be trained on a compressed representation of environment observations paired with a predictive model of latent dynamics. Dreamer – which we leverage in this work – extends this approach by training a world model and actor jointly via short-horizon prospection for long-horizon control (Hafner et al., 2019, 2025). These models provide an efficient computational implementation of prospection by predicting the consequences of candidate actions in a proxy representation of the environment.

An emerging world model design called JEPA (LeCun et al., 2022) predicts future states directly in a learned latent space, encouraging representations that preserve information needed for prospection. A more recent variant (Destrade et al., 2025) shapes the world model's representation with goal-conditioned value functions, crucial for anticipating failures. While the primary use case for world models has been for direct planning or solver optimization, we leverage world-model prospection within an AT process and shape the agent's adaptation towards explicit risk mitigation.

Continual Learning and Task Discovery: Continual learning crucially balances adaptation to new tasks with preservation of critical-task competence by leveraging replay, regularization, and related mechanisms to limit forgetting (Wang et al., 2024). PowerPlay defines a framework that enables continual learning by autonomously expanding solver abilities through the identification of the simplest tasks just beyond the current boundary of competence while preserving prior capabilities (Schmidhuber, 2013). Our approach draws inspiration from this framework, using world-model prospection to identify nearby, risk-relevant failures and maintaining a mixed replay of solved scenarios and discovered failures.

5. Conclusion

The AT needed to manage risks in safety- and mission-critical tasks is a demanding form of complex cognition. It requires enough domain expertise to formulate new problems that are derived from a critical task and then generate a solution that solves the new task while ensuring that the critical task can still be solved. We defined a computational model of AT to address this complexity and in doing so provided the framework for AI agents to address application risks in an explicitly computable manner. The steps of our AT model (prospection, creativity, adaptation) can be investigated independently and be of different representations, nimbly tuning the risk profiles and providing assurances needed to unlock the type of decision support for managing risk in the long-tail-of-errors.

We have leveraged the emergence of world models to fill an important gap, prospection, in enabling AT for AI agents with sub-symbolic representations and catalyzing a generalized computational model of AT. World models enable prospection by providing an efficient offline understanding of the environment in which an agent operates, allowing it to predict possible catastrophic futures the agent has not yet encountered and adapt to them ahead of time.

We operationalized our computational model of AT as a reinforcement learning agent that led to three key results. First, we show that investing in AT is a worthwhile endeavor to manage risks. Generating stress-tests from critical tasks ensures a margin of safety on real-world task performance, rather than attempting to execute those tasks directly without accounting for long-tail risks. This is evident from the catastrophic performance drop when Baseline-Dreamer encounters new track widths and AT-Dreamer suffers only slight performance degradation. Second, using AT improves overall performance. In this intriguing result, we show that when adapting to stress-tests, our AT-Dreamer agent acquires knowledge useful to improve its performance on tasks it has already solved. While unexpected, it reflects a way in which humans improve their mental models of the world. Third, the systematic curriculum generated from AT is more effective than simply increasing training data. This approach introduces an experience-based form of sample efficiency by explicitly finding the highest-value samples for the agent's adaptation: catastrophic policy failures in states closest to the agent's current competence boundary. Based on PowerPlay (Schmidhuber, 2013) learning, our results show an agent that continually tests this boundary acquires policies that transfer beyond the particular failures used during training.

The experiments presented here demonstrate an example operationalization of AT in reinforcement learning. Future approaches could derive prospection from other predictive architectures, including JEPA-style models augmented with task-relevant value, uncertainty, or prediction-error measures (Destrade et al., 2025; Assran et al., 2023; LeCun et al., 2022). Adaptation could use parameter-efficient adapters (Houlsby et al., 2019), meta-learned fast adaptation (Finn et al., 2017), or continual-learning methods designed to incorporate new failures while limiting catastrophic forgetting (Kirkpatrick et al., 2017; Lopez-Paz & Ranzato, 2017). Creativity could similarly move beyond grid search to methods such as the cross-entropy method or gradient-based adversarial search (Ren & Majumdar, 2022). At a higher level, LLM agents could apply structured analytic techniques (Heuer, 1999) to propose novel task variations and select prospective metrics appropriate to the environment and available world model. Future agents should not merely learn from the problems they encounter; they should actively construct problems that reveal what they have not yet learned.

6. Limitations

The formal definition for our computational model requires validation across additional representations. Although recent advances in world models enabled us to operationalize AT in reinforcement learning, other representations could be recast onto our model, such as previous AT goal reasoning work from Amos-Binks & Dannenhauer (2020). Furthermore, we do not specify the requirements for homogeneous representations across the three steps in an AT cycle. Our goal is for AI systems to have the mechanism to make assurances on managing risks. We hypothesize that, for some applications, mixed-representation systems would accomplish this goal. A wider range of representations across symbolic to subsymbolic would strengthen this model's generalization claim.

In demonstrating our computational model, we chose the CarRacing domain for its simplicity. It provides established or straightforward-to-implement baseline agents while remaining intuitive to those without an RL background. Evaluating across a wider range of RL domains would provide more convincing evidence of the value of AT in more complex classical RL problems (i.e., those with large action spaces such as StarCraft 2) as well as those with mission critical properties (e.g., the X-plane flight simulator).

Lastly, our work is limited by a lack of understanding of when a world model is sufficiently performant to support AT. Without this theoretical grounding, we do not know a priori if the step up capability of AT is feasible to achieve in any given domain. We hypothesize that there are classes of domains where training or developing a world model is unfeasible and AT may not provide the step-up performance improvement.

Acknowledgements

This work is supported by the US Army Medical Research and Development Command under Contract No. HT9425-24-9-0042. The opinions or assertions contained herein are the private ones of the authors/speaker and are not to be construed as official or reflecting the views of the Department of Defense or any other agency of the U.S. Government.

References

Amos-Binks, A., & Dannenhauer, D. (2020). Anticipatory Thinking: A metacognitive capability. *AAAI Fall Symposium on Cognitive Systems for Anticipatory Thinking - CEUR Workshop Proceedings*, *2558*.

Amos-Binks, A., Dannenhauer, D., & Gilpin, L. H. (2023). The anticipatory paradigm. *AI Magazine*, *44*, 133–143.

Amos-Binks, A., & Weber, B. S. (2023). Risk Management: Anticipating and Reacting in StarCraft. *Proceedings - AAAI Artificial Intelligence and Interactive Digital Entertainment Conference, AIIDE*, *19*, 13–22.

Argenta, C., Browning, A., Amos-binks, A., & Lyle, M. (2020). Avoiding Surprise : Augmenting Anticipatory Thinking with Scenario Explorer. *AAAI Fall Symposium on Anticipatory Thinking*.

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-supervised learning from images with a joint-embedding predictive architec-

ture. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 15619–15629).

Destrade, M., Bounou, O., Lidec, Q. L., Ponce, J., & LeCun, Y. (2025). Value-guided action planning with jepa world models. *arXiv preprint arXiv:2601.00844*.

Ding, J., et al. (2025). Understanding world or predicting future? a comprehensive survey of world models. From https://doi.org/10.1145/3746449.

Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. *International conference on machine learning* (pp. 1126–1135). PMLR.

Geden, M., Smith, A., Campbell, J., Spain, R., Amos-Binks, A., Mott, B., Feng, J., & Lester, J. (2019). Construction and Validation of an Anticipatory Thinking Assessment. *Frontiers in Psychology*, *10*, 1–10.

Ha, D., & Schmidhuber, J. (2018). World Models. In *Neurips*.

Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2019). Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*.

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. *Nature*, *640*, 647–653.

Heuer, R. (1999). *Psychology of Intelligence Analysis*.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for nlp. *International conference on machine learning* (pp. 2790–2799). PMLR.

InexperiencedMe (2025). NaturalDreamer: A simple and clean implementation of DreamerV3. https://github.com/InexperiencedMe/NaturalDreamer. GitHub repository, accessed 2026-07-15.

Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, *114*, 3521–3526.

Klein, G., Snowden, D., & Pin, C. L. (2011). Anticipatory Thinking. In *Informed by knowledge: Expert performance in complex situations*, chapter 15, 235–246.

LeCun, Y., et al. (2022). A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, *62*, 1–62.

Lopez-Paz, D., & Ranzato, M. (2017). Gradient episodic memory for continual learning. *Advances in neural information processing systems*, *30*.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of machine learning research*, *22*, 1–8.

Ren, A. Z., & Majumdar, A. (2022). Distributionally robust policy learning via adversarial environment generation. *IEEE Robotics and Automation Letters*, *7*, 1379–1386.

Ritchie, G. (2001). Assessing creativity. *AISB'01 Symposium on Artificial Intelligence and Creativity in Arts and Science*. From http://scholar.google.com/scholar?hl=en&btnG=Search&q=intitle:Assessing+Creativity#2.

Schacter, D. L., Benoit, R. G., & Szpunar, K. K. (2017). Episodic future thinking: mechanisms and functions. *Current Opinion in Behavioral Sciences*, *17*, 41–50. From http://dx.doi.org/10.1016/j.cobeha.2017.06.002.

Schmidhuber, J. (2013). Powerplay: Training an increasingly general problem solver by continually searching for the simplest still unsolvable problem. *Frontiers in psychology*, *4*, 313.

Szpunar, K. K., Spreng, R. N., & Schacter, D. L. (2014). A taxonomy of prospection: Introducing an organizational framework for future-oriented cognition. *Proceedings of the National Academy of Sciences*, *111*, 18414–18421. From http://www.pnas.org/lookup/doi/10.1073/pnas.1417144111.

Wang, L., Zhang, X., Su, H., & Zhu, J. (2024). A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*, 5362–5383. From https://doi.org/10.48550/arxiv.2302.00487.